

ENERGIA PARA SOBREVIVER

O CÁLCULO
DIFERENCIAL QUE
MODELA AS
VARIACÕES DO
SISTEMA
ENERGETICO

4

LISTA DE FIGURAS

Figura 1 - Visualização de uma função racional $f(x) = (x+1)/(x-1)$ na região próxima de $x=1$	8
Figura 2 – Expressão representando limite	9
Figura 3 - Visualização de um limite de $f(x) = x^3 - 2x^2 + x + 1$ com x tendendo a 1,5	9
Figura 4 – Função contínua.....	10
Figura 5 – Limites conhecidos de funções básicas	11
Figura 6 – Propriedades de limites de funções	12
Figura 7 – Expressão derivada.....	13
Figura 8 - Interpretação geométrica da derivada através da reta secante (em cima) e da reta tangente (meio) em um ponto do gráfico. O gráfico da derivada (embaixo) representa a inclinação da reta tangente nos pontos da função original.....	15
Figura 9 – Exemplos de derivadas conhecidas	16
Figura 10 – Exemplos de propriedades de derivadas	17
Figura 11 - Função $f(x) = (3x^2 + 1) \ln(x)$ e sua derivada	19
Figura 12 - Função valor absoluto e sua derivada (degrau unitário)	20
Figura 13 – Diferentes notações para derivadas.....	21
Figura 14 – Dois exemplos de quartas derivadas de funções.....	21
Figura 15 - Derivadas numéricas de ordem superior	22
Figura 16 - Pontos críticos de uma função polinomial.....	24
Figura 17 – Informações que podem ser obtidas a partir da primeira e da segunda derivada	25
Figura 18 - Avaliação da primeira e da segunda derivada de x^3 em $x=0$	26
Figura 19 – Limite com o número de Euler	28
Figura 20 - Relação entre posição, velocidade e aceleração em um movimento uniformemente variado.....	30
Figura 21 - Visualização do limite de $\sin(x)/x$ para $x \rightarrow 0$ e solução com L'Hôpital.....	31
Figura 22 - Linearização da função raiz quadrada de x com Taylor de 1ª ordem	32
Figura 23 - Otimização com gradiente descendente obtendo um mínimo local	34
Figura 24 - Gráfico de contornos (em cima) e gráfico da superfície durante minimização com gradiente descendente (embaixo)	36
Figura 25 - Minimização de função com SciPy.....	38
Figura 26 - Ajuste de $f(x)$ linear via gradiente descendente	40
Figura 27 - Ajuste de $f(x)$ na forma de parábola via gradiente descendente	41

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Comparação da computação numérica (NumPy) e simbólica (SymPy).....	6
Código-fonte 2 – Declaração de variáveis simbólicas com SymPy	6
Código-fonte 3 – Visualização das expressões matemáticas de SymPy com LaTeX	7
Código-fonte 4 – Manipulação algébrica de polinômios com SymPy	7
Código-fonte 5 – Obtenção de limite com SymPy	9
Código-fonte 6 – Avaliação de limites da função racional escolhida	10
Código-fonte 7 – Avaliação de propriedades de limites com SymPy	12
Código-fonte 8 – Avaliação de propriedades de limites com SymPy.	13
Código-fonte 9 – Limites de funções básicas com SymPy	17
Código-fonte 10 – Propriedades de derivadas com SymPy	17
Código-fonte 11 – Derivada numérica implementada com numpy.gradient	20
Código-fonte 12 - Derivadas de ordem superior implementadas com numpy.gradient.	22
Código-fonte 13 – Derivadas de ordem superior implementadas com SymPy	22
Código-fonte 14 – Código para localizar pontos críticos de uma função	23
Código-fonte 15 – Código de geração do limite	29
Código-fonte 16 – Conversão de variáveis entre SymPy e NumPy	30
Código-fonte 17 – Exemplo do uso da regra de L'Hôpital na solução de limites na forma indeterminada	31
Código-fonte 18 – Aproximação de Taylor de primeira ordem usando expressões analíticas	32
Código-fonte 19 – Implementação de gradiente descendente 1D com derivada numérica.....	34
Código-fonte 20 – Minimização de funções com scipy.optimize.minimize	37
Código-fonte 21 – Uso de diferenciação automática para cálculo de gradientes	39
Código-fonte 22 – Comparação de resultados de diferenciação com abordagem numérica, simbólica e <i>autodiff</i>	39
Código-fonte 23 – Ajuste de $f(x)$ linear via gradiente descendente	40
Código-fonte 24 – Ajuste de $f(x)$ na forma de parábola via gradiente descendente ..	41

SUMÁRIO

CÁLCULO DIFERENCIAL APLICADO.....	5
1 INTRODUÇÃO	5
1.1 Computação simbólica com SymPy	6
2 LIMITES DE FUNÇÕES	7
2.1 Propriedades e limites de funções básicas	10
3 DERIVADAS DE FUNÇÕES DE UMA VARIÁVEL	13
3.1 Interpretação geométrica da derivada	14
3.2 Propriedades e derivadas das funções básicas	16
3.3 Funções que não são diferenciáveis em todos os pontos	19
3.4 Derivadas de ordem superior	20
3.5 Pontos críticos	23
4 APLICAÇÕES DE LIMITES E DERIVADAS.....	26
4.1 Juros compostos e a constante de Euler com limites.....	26
4.2 Diferenciação simbólica com SymPy com aplicação na física	29
4.3 Regra de L'Hôpital.....	30
4.4 Expansão de Taylor de primeira ordem	31
4.5 Otimização com gradiente descendente: Casos 2D e 3D	33
4.6 Otimização com SciPy.....	37
4.7 Diferenciação automática com JAX e ajuste de curvas.....	38
5 CONCLUSÃO.....	41
REFERÊNCIAS.....	43
GLOSSÁRIO	44

CÁLCULO DIFERENCIAL APLICADO

1 INTRODUÇÃO

A matemática oferece uma linguagem adequada para descrever fenômenos da natureza. Nós identificamos grandezas que podem ser medidas e buscamos relações entre elas para formar modelos matemáticos, que são representações idealizadas que buscam preservar as características essenciais, organizando e sistematizando as observações.

Anteriormente, utilizamos funções matemáticas para descrever modelos, focando nas expressões algébricas e gráficos que elas produzem. Muitos dos problemas buscam compreender fenômenos que mudam ao longo do tempo. Vamos considerar um exemplo da física. Imagine observar um carro em movimento e tentar descrevê-lo apenas com números. Ao anotarmos a posição em diferentes momentos, obtemos uma lista de valores que pode ficar igual, crescer ou diminuir com o passar do tempo; isso não é muito prático. Uma alternativa é representar esse movimento com funções matemáticas. Assim, a sua posição passa a ser descrita por uma função do tempo: $x(t)$.

É importante saber onde o carro está, mas dificilmente ele vai manter a velocidade durante seu trajeto. Essa é uma informação que queremos saber: se sua velocidade $v(t)$ é sempre a mesma ou se ela aumenta ou diminui. Simultaneamente, para que a velocidade varie, precisamos saber se o carro está acelerando ou desacelerando com uma aceleração $a(t)$.

Essa situação ilustra que existem funções que são relacionadas com outras, ou seja, as formas de $a(t)$, $v(t)$ e $x(t)$ devem guardar algum vínculo pelo modo como variam no tempo. Mais do que isso, esse ponto de vista nos mostra a importância de investigar padrões de variação de funções e seus detalhes.

Ao longo desse capítulo, vamos desenvolver a linguagem necessária para descrever tais relações através de dois conceitos essenciais. O primeiro é o de limite, quando deixaremos de perguntar apenas quanto vale a função em um ponto específico para investigar de que valor a função se aproxima conforme chegamos perto desse ponto. Já o segundo é o de derivada, quando avaliaremos qual é a taxa

de variação instantânea de uma função e como isso pode ser utilizado para otimizar problemas na busca pela melhor solução possível.

1.1 Computação simbólica com SymPy

Em todos os exemplos até agora, temos trabalhado com computação numérica, utilizando bibliotecas como `NumPy` que aproximam os valores verdadeiros. Números irracionais como o π são apresentados e utilizados com um número finito de casas decimais, de modo que erros numéricos são inevitáveis.

Uma alternativa está na computação simbólica, que traz uma representação exata dos objetos matemáticos. Em Python, a biblioteca `SymPy` traz essa abordagem, que será bastante explorada nesse material. Para ilustrar, o código-fonte a seguir compara operações equivalentes em `NumPy` e `SymPy`. Enquanto a primeira mostra resultados aproximados, a segunda mostra resultados exatos.

```
import numpy as np
print(np.sqrt(9))
print(np.sqrt(8))
print(np.pi)
import sympy as sp
print(sp.sqrt(9))
print(sp.sqrt(8))
print(sp.pi)
```

Código-fonte 1 – Comparação da computação numérica (NumPy) e simbólica (SymPy)
Fonte: Elaborado pelo autor (2025)

No exemplo anterior, foram visualizados diretamente valores de constantes ou números reais. Podemos também definir símbolos e utilizá-los para definir variáveis. O código-fonte abaixo mostra duas formas diferentes de realizar essa funcionalidade.

```
import sympy as sp
x, a, b = sp.symbols("x a b")
print(a*x + b)
from sympy.abc import y, c, d
print(c*y + d)
```

Código-fonte 2 – Declaração de variáveis simbólicas com SymPy
Fonte: Elaborado pelo autor (2025)

Ressalta-se que o `SymPy` tem integração com LaTeX, um sistema para produção de documentos com linguagem de marcação que permite escrever expressões matemáticas de uma forma prática e eficiente. O código-fonte a seguir

mostra como definir uma matriz simbolicamente, além de demonstrar o seu trecho de código em LaTeX e sua exibição renderizada.

```
from sympy import latex
from IPython.display import Math
import sympy as sp
sp.init_printing(use_latex='mathjax')

a11, a12, a13 = sp.symbols("a_{11} a_{12} a_{13}")
a21, a22, a23 = sp.symbols("a_{21} a_{22} a_{23}")
a31, a32, a33 = sp.symbols("a_{31} a_{32} a_{33}")

Sym = sp.Matrix([[a11, a12, a13],
                  [a12, a22, a23],
                  [a13, a23, a33]])

print(latex(Sym))
Math(latex(Sym))
```

Código-fonte 3 – Visualização das expressões matemáticas de SymPy com LaTeX
Fonte: Elaborado pelo autor (2025)

Apenas para ilustrar o uso de SymPy além da representação, para realização de operações, o código-fonte abaixo traz algumas operações básicas com polinômios, como fatoração, expansão e o cálculo de raízes. Ao longo do capítulo, diversas outras operações serão mostradas e discutidas.

```
import sympy as sp
x = sp.Symbol('x')          # declara uma variável simbólica
f = x**2 + 2*x + 1          # constrói a expressão (x + 1)^2

factored = sp.factor(f)     # fatora a expressão
print(factored)             # (x + 1)**2

expanded = sp.expand(factored) # expande a expressão
print(expanded)             # x**2 + 2*x + 1

print(sp.solve(f, x))
print(sp.roots(f, x))
```

Código-fonte 4 – Manipulação algébrica de polinômios com SymPy
Fonte: Elaborado pelo autor (2025)

2 LIMITES DE FUNÇÕES

Em matemática, o conceito de limite de uma função é fundamental para entender o que acontece com os valores da função quando a variável se aproxima de um ponto, sem necessariamente alcançá-lo. Em alguns casos, a resposta pode até

parecer óbvia. Se tivermos a equação de uma reta do tipo $f(x) = ax+b$ e quisermos avaliar a função em $x=5$, a substituição direta já nos traz o resultado.

Outro exemplo é o de uma função racional $f(x) = (x+1)/(x-1)$, cuja visualização é mostrada na figura abaixo. Quando x se aproxima de 1, o denominador vai para 0, o que causa um comportamento diferente dependendo da direção que estamos considerando. Se viermos da direita para a esquerda, de números maiores do que 1, o gráfico crescerá indefinidamente perto de $x=1$. Se viermos da esquerda para a direita, de números menores do que 1, o gráfico decrescerá indefinidamente. Em outras palavras, não podemos de fato substituir $x=1$ nessa função, mas podemos avaliar o seu comportamento em torno desse valor.

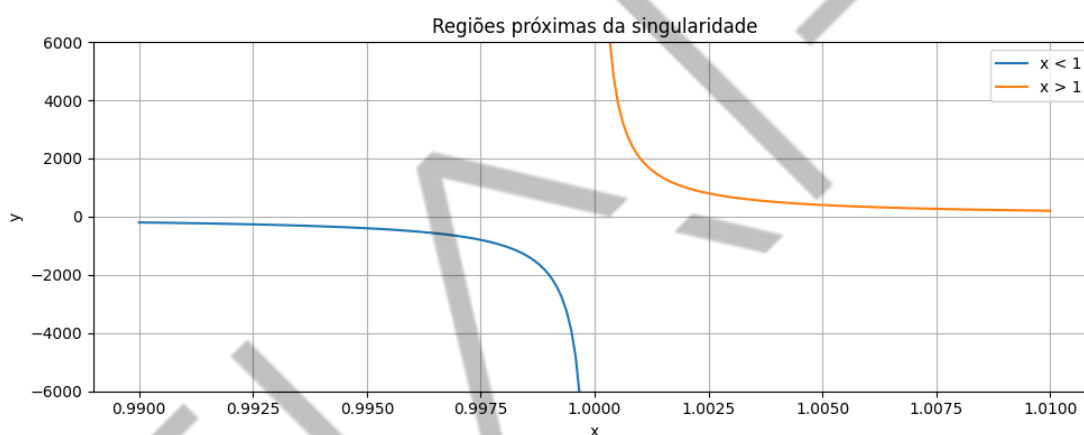


Figura 1 - Visualização de uma função racional $f(x) = (x+1)/(x-1)$ na região próxima de $x=1$
Fonte: Elaborada pelo autor (2025)

São diversos os casos relevantes. Com funções trigonométricas, por exemplo, sempre que o cosseno $\cos(x) = 0$, a tangente $\tan(x)$ deixa de estar definida, mas podemos avaliá-la nas proximidades desses pontos. Da mesma forma, ao avaliar o comportamento de funções quando a variável independente x vai para o infinito, nem sempre isso significa que a função também resulte em infinito. Limites têm papel necessário na avaliação desses e de muitos outros casos.

Em termos de notação, utiliza-se a expressão abaixo, na qual **lim** representa limite, **f(x)** é a função que está sendo avaliada, **x→p** significa que a variável independente **x** está tendendo ao valor **p** e isso resulta no valor **L** para **f(x)**. Essa expressão explicita que o resultado de um limite é um número.

$$\lim_{x \rightarrow p} f(x) = L$$

Figura 2 – Expressão representando limite
Fonte: Elaborada pelo autor (2025)

Para calcular limites de funções, a forma mais formal é através do que é conhecido como a definição (ϵ , δ). Como a ênfase do capítulo é computacional, ela não será detalhada. Com tal objetivo, será utilizado o `SymPy`. O código-fonte abaixo traz um exemplo de limite de um polinômio com $x \rightarrow 1.5$, ilustrado na figura “Visualização de um limite de $f(x) = x^3 - 2x^2 + x + 1$ com x tendendo a 1,5”.

```
from sympy import Limit, Symbol
import numpy as np
# definir a função
x = Symbol('x')
f = (x**3 - 2*x**2 + x + 1)
ponto = 1.5
# calcular o limite quando x tende a 1.5
limite = Limit(f, x, ponto).doit()
print("O limite da função quando x tende a 1.5 é:", limite)
```

Código-fonte 5 – Obtenção de limite com SymPy
Fonte: Elaborado pelo autor (2025)

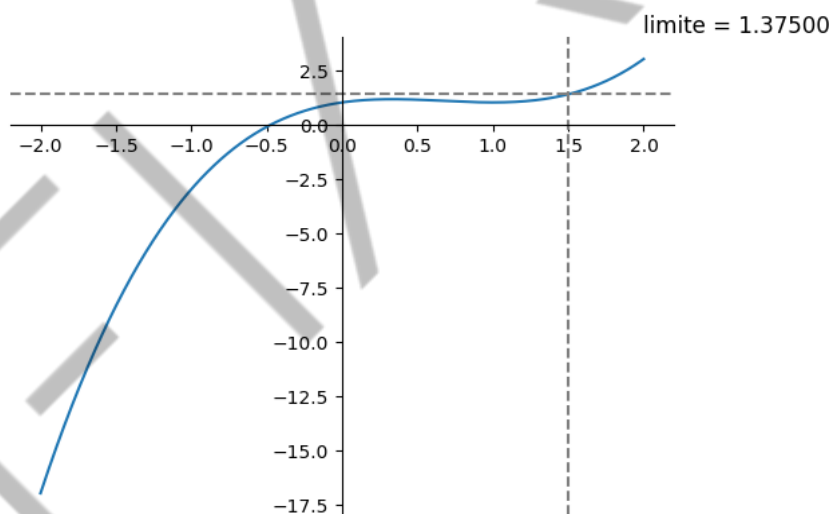


Figura 3 - Visualização de um limite de $f(x) = x^3 - 2x^2 + x + 1$ com x tendendo a 1,5
Fonte: Elaborada pelo autor (2025)

Diferentemente da figura “Visualização de uma função racional $f(x) = (x+1)/(x-1)$ na região próxima de $x=1$ ”, quando o comportamento da função era diferente dependendo do lado que você se aproximasse do ponto, na figura “Visualização de um limite de $f(x) = x^3 - 2x^2 + x + 1$ com x tendendo a 1,5” o valor de $f(x)$ é o mesmo seja pela esquerda ou pela direita, ou seja, sempre devemos avaliar os dois limites laterais de uma função em um determinado ponto p . Se os resultados forem iguais,

então existe o limite bilateral de $f(x)$ em p , ou simplesmente limite. Se os limites laterais forem diferentes ou não existirem, o limite não existe no ponto p .

A existência dos dois limites laterais de $f(x)$ em um ponto específico é diretamente relacionada com a definição de continuidade de funções. Uma função contínua em um ponto é aquela que:

$$\lim_{x \rightarrow c} f(x) = f(c)$$

Figura 4 – Função contínua
Fonte: Elaborada pelo autor (2025)

Em outras palavras, ela é contínua no ponto se o limite bilateral existe naquele ponto.

Para ilustrar essa discussão com `SymPy`, o próximo código-fonte avalia diversos limites da função racional da figura “Visualização de um limite de $f(x) = x^3 - 2x^2 + x + 1$ com x tendendo a 1,5”. Vale notar que, quando os limites laterais divergem para infinitos de sinais opostos, o limite bilateral não existe, o que o `SymPy` representa pelo símbolo “zoo”.

```
import sympy as sp
x = sp.symbols('x')
f = (x+1)/(x-1)
print(sp.limit(f, x, 2))
print(sp.limit(f, x, 1))
print(sp.limit(f, x, 1, dir='+'))
print(sp.limit(f, x, 1, dir='-'))
print(sp.limit(f, x, 1, dir='+-'))
print(sp.limit(f, x, sp.oo))
print(sp.limit(f, x, -sp.oo))
```

Código-fonte 6 – Avaliação de limites da função racional escolhida
Fonte: Elaborado pelo autor (2025)

2.1 Propriedades e limites de funções básicas

Diversos resultados de propriedades e limites já foram verificados. As figuras “Limites conhecidos de funções básicas” e “Propriedades de limites de funções” apresentam alguns desses resultados. Quando queremos abordar limites de expressões com muitos termos, podemos identificar quais são as propriedades e funções necessárias e avaliar quais são os resultados que podem ser diretamente

utilizados. Ressalta-se que aqui o conhecimento e uso da ferramenta é priorizado, enquanto posteriormente serão discutidos exemplos práticos.

De modo geral, a atenção deve ser dada quando a substituição direta de $x \rightarrow p$ leva a resultados como $0/0$, $\infty/0$ ou ∞/∞ , as chamadas formas indeterminadas. Não é possível dizer de imediato que tais limites não existem. Antes, é necessário tentar simplificar as expressões com manipulações algébricas.

Por exemplo, seja o limite de $(x^3 - 3x^2 + x - 3)/(x - 3)$, com $x \rightarrow 3$. A substituição direta levaria ao resultado $0/0$. Identificando que o numerador pode ser escrito como $(x - 3)(x^2 + 1)$, é possível simplificar o problema e calcular apenas $\lim_{x \rightarrow 3} (x^2 + 1)$. O resultado é 10, agora sim obtido pela substituição direta. Nesse caso, a solução parte do conhecimento e manipulação de polinômios, não algo específico de limites.

O código-fonte “Avaliação de propriedades de limites com SymPy” ilustra o limite de algumas funções básicas e a avaliação de limites laterais pelos dois lados, bem como limites com x tendendo ao infinito.

Observação: nesse exemplo e em outros, os resultados foram armazenados em variáveis, mas poderiam ser visualizados diretamente com `print()`.

Tipo de limite	Expressão matemática	Resultado	Observação
Limite da identidade	$\lim_{x \rightarrow a} x$	a	Substituição direta
Limite de constante	$\lim_{x \rightarrow a} c$	c	Constante não depende de x
Limite de potência	$\lim_{x \rightarrow a} x^n$	a^n	$n \in \mathbb{N}$
Limite de polinômio	$\lim_{x \rightarrow a} P(x)$	$P(a)$	Sempre contínuos
Limite trigonométrico	$\lim_{x \rightarrow a} \sin(x)$	$\sin(a)$	Função contínua
Limite no infinito	$\lim_{x \rightarrow \infty} \frac{1}{x}$	0	Comportamento assintótico
Limite infinito	$\lim_{x \rightarrow \infty} x^2$	∞	Crescimento sem limite

Figura 5 – Limites conhecidos de funções básicas

Fonte: Elaborada pelo autor (2025)

```
from sympy import symbols, limit, oo, sin, Piecewise
x = symbols('x')
# Limites de funções básicas
lim_x = limit(x, x, 2)           # lim x->2 (x) = 2
lim_x2 = limit(x**2, x, 3)       # lim x->3 (x^2) = 9
lim_x2_inf = limit(x**2, x, oo)  # lim x->∞ (x^2) = ∞
```

```
lim_inf_pos = limit(1/x, x, oo) # lim x->∞ (1/x) = 0
lim_inf_neg = limit(1/x, x, -oo) # lim x->-∞ (1/x) = 0
# lim x->∞ (2x^2+3x)/(x^2-x) = 2
lim_pol_inf = limit((2*x**2 + 3*x) / (x**2 - x), x, oo)
```

Código-fonte 7 – Avaliação de propriedades de limites com SymPy
Fonte: Elaborado pelo autor (2025)

As principais propriedades de limites são apresentadas na figura “**Propriedades de limites de funções**”, enquanto exemplos específicos são realizados com SymPy no código-fonte a seguir.

Em um curso de cálculo convencional, a abordagem ideal é a que resolver diferentes exercícios de limites com base nas propriedades e teoremas conhecidos. Os livros de referência ao final desse capítulo discutem isso. Por aqui, a ênfase maior está justamente na conexão do conceito matemático com a sua representação computacional, mas as duas perspectivas podem ser aprofundadas. Pode-se verificar cada exemplo “na mão”, bem como partir para uma perspectiva de computação numérica, avaliando funções em pontos específicos e gerando gráficos para entender os limites.

Propriedade	Expressão matemática	Resultado	Condição / Observação
Soma	$\lim_{x \rightarrow a} [f(x) + g(x)]$	$\lim_{x \rightarrow a} f(x) + \lim_{x \rightarrow a} g(x)$	Ambos os limites existem
Diferença	$\lim_{x \rightarrow a} [f(x) - g(x)]$	$\lim_{x \rightarrow a} f(x) - \lim_{x \rightarrow a} g(x)$	Ambos os limites existem
Produto	$\lim_{x \rightarrow a} [f(x) \cdot g(x)]$	$\lim_{x \rightarrow a} f(x) \cdot \lim_{x \rightarrow a} g(x)$	Ambos os limites existem
Quociente	$\lim_{x \rightarrow a} \frac{f(x)}{g(x)}$	$\frac{\lim_{x \rightarrow a} f(x)}{\lim_{x \rightarrow a} g(x)}$	$\lim_{x \rightarrow a} g(x) \neq 0$
Constante×função	$\lim_{x \rightarrow a} [c \cdot f(x)]$	$c \cdot \lim_{x \rightarrow a} f(x)$	c constante
Valor absoluto	$\lim_{x \rightarrow a} f(x) $	$ \lim_{x \rightarrow a} f(x) $	Limite existe
Função composta	$\lim_{x \rightarrow a} f(g(x))$	$f(\lim_{x \rightarrow a} g(x))$	f contínua em $\lim g(x)$

Figura 6 – Propriedades de limites de funções
Fonte: Elaborada pelo autor (2025)

```
from sympy import symbols, limit, oo, sin, Piecewise
x = symbols('x')# Propriedades de limites
f = x
g = x**2
c = 5
lim_soma = limit(f + g, x, 2) # lim(f + g) = lim f + lim g
lim_produto = limit(f * g, x, 2) # lim(f * g) = lim f * lim g
lim_quociente = limit(f / g, x, 2) # lim(f / g) = lim f / lim g
```

```

lim_constante = limit(c, x, 2)           # lim(constante) = constante
lim_multiplo = limit(c * f, x, 2)        # lim(c*f) = c * lim f

# Limite de função composta
h = sin(x**2)                            # f(g(x)) = sin(x^2)
lim_composta = limit(h, x, 0)            # lim x->0 sin(x^2) = 0

# Limites laterais
# Função em pedaços para exemplo de descontinuidade
f_pieewise = Piecewise((x, x < 1), (2*x, x >= 1))
# limite à esquerda lim x->1^- f(x) = 2
lim_esq = limit(f_pieewise, x, 1, dir='-')
# limite à direita lim x->1^+ f(x) = 2
lim_dir = limit(f_pieewise, x, 1, dir='+')

```

Código-fonte 8 – Avaliação de propriedades de limites com SymPy.

Fonte: Elaborado pelo autor (2025)

3 DERIVADAS DE FUNÇÕES DE UMA VARIÁVEL

A derivada é um operador matemático que permite descrever quantitativamente a variação de uma função. Ela indica se uma função está aumentando ou diminuindo ao longo do domínio e com que intensidade essa variação ocorre. Também permite comparar taxas de crescimento entre diferentes funções, identificando quais crescem mais rapidamente e quais apresentam crescimento lento. Esse conceito também permite distinguir comportamentos assintóticos, como funções cujo crescimento é gradual, a exemplo do logaritmo, em contraste com funções cujo crescimento é acelerado, como exponenciais ou certas funções racionais.

Formalmente, define-se a derivada como o limite a seguir. Ressalta-se que o apóstrofo em $f'(x)$ é uma das possíveis notações de derivada.

$$f'(x) = \lim_{\Delta x \rightarrow 0} \frac{f(x + \Delta x) - f(x)}{\Delta x}$$

Figura 7 – Expressão derivada
Fonte: Elaborada pelo autor (2025)

Essa expressão tem algumas características diferentes de um limite como vimos anteriormente. Do lado esquerdo há uma função de x , ou seja, a derivada de uma função é uma derivada que pode ser utilizada para calcular o valor em um ponto específico, mas não necessariamente. Vamos recordar que os limites que vimos

anteriormente eram calculados em pontos específicos, resultando em valores numéricos fixos.

Do lado direito, temos uma expressão no numerador que indica uma diferença entre pontos de $f(x)$. Devemos avaliá-la em um ponto inicial x e em um ponto com um incremento $x + \Delta x$. A diferença entre eles indica quanto o valor da variável dependente $y = f(x)$ mudou para um determinado intervalo da variável independente x . Ao mesmo tempo, vemos que o limite é calculado para o incremento Δx tendendo a zero, ou seja, estamos avaliando intervalos cada vez menores.

Com base nessas características, podemos afirmar que a derivada é um operador que atua sobre uma função e produz outra função, indicando a proporção da mudança de y em relação à mudança de x . Isso é realizado calculando como a função varia em intervalos cada vez menores até obter uma taxa de variação instantânea no limite.

3.1 Interpretação geométrica da derivada

A interpretação geométrica da derivada surge diretamente da análise do gráfico da função. Quando calculamos a diferença $f(x + \Delta x) - f(x)$ para um valor não-nulo de Δx , estamos identificando dois pontos no gráfico e traçando uma reta secante que passa por ambos. A inclinação dessa reta representa a taxa de variação média da função naquele intervalo. Isso é mostrado na parte de cima da figura a seguir.

Ao aplicarmos o limite de $\Delta x \rightarrow 0$, forçamos o segundo ponto a se aproximar do primeiro até que a distância entre eles seja infinitesimal. Nesse processo, a reta secante se movimenta e se ajusta à curvatura da função, transformando-se em uma reta tangente. Essa reta toca a curva em apenas um ponto e sua inclinação é o valor numérico da derivada naquele local. Em outras palavras, o valor da derivada naquele ponto é o coeficiente angular da reta tangente. Isso é mostrado na parte do meio da figura.

A parte de baixo da imagem também traz o gráfico da derivada $f'(x)$ em si, o que permite rapidamente avaliar como a função original $f(x)$ está variando.

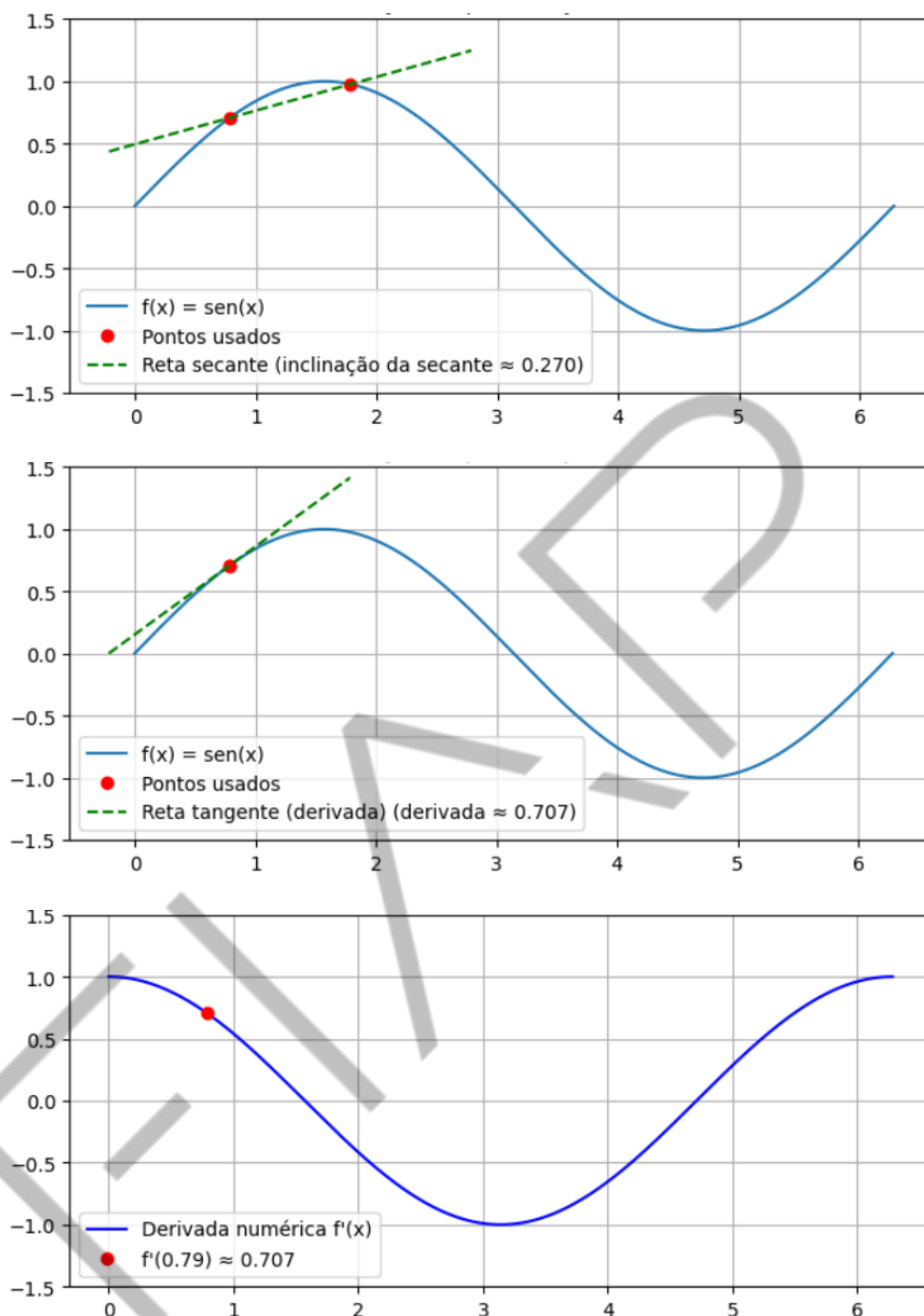


Figura 8 - Interpretação geométrica da derivada através da reta secante (em cima) e da reta tangente (meio) em um ponto do gráfico. O gráfico da derivada (embaixo) representa a inclinação da reta tangente nos pontos da função original

Fonte: Elaborada pelo autor (2025)

Essa taxa de variação instantânea da curva traz informações de intensidade e direção que são importantes. Em relação à intensidade, quanto maior, em módulo, o valor, mais rapidamente a função varia.

Em relação à direção dessa variação:

- Se o sinal da derivada é positivo, $f'(x) > 0$, a função está crescendo;
- Se é negativo, $f'(x) < 0$, a função está decrescendo;
- Se a derivada se anula, $f'(x) = 0$, a função apresenta uma taxa de variação nula, o que caracteriza pontos onde a curva 'estaciona' antes de mudar de direção ou seguir uma tendência.

3.2 Propriedades e derivadas das funções básicas

Existem muitos resultados que já foram obtidos a partir da definição de derivadas baseada em limites. Aqui eles são mostrados como forma de material de referência. Na sequência, utilizaremos tais resultados em exemplos.

Na figura abaixo são apresentadas algumas derivadas conhecidas, enquanto o código-fonte “Limites de funções básicas com SymPy” traz alguns exemplos com SymPy.

Função original	Resultado da derivada
$f(x) = c$	0
$f(x) = x$	1
$f(x) = x^n$	nx^{n-1}
$f(x) = e^x$	e^x
$f(x) = a^x$	$a^x \ln a$
$f(x) = \ln x$	$\frac{1}{x}$
$f(x) = \sin x$	$\cos x$
$f(x) = \cos x$	$-\sin x$
$f(x) = \tan x$	$\sec^2 x$

Figura 9 – Exemplos de derivadas conhecidas
Fonte: Elaborada pelo autor (2025)

```
import sympy as sp
x = sp.Symbol('x')
print("d/dx (c) =", sp.diff(5, x)) # Derivada de uma constante
print("d/dx (x^2) =", sp.diff(x**2, x)) # Função quadrática
print("d/dx (x^3) =", sp.diff(x**3, x)) # Função cúbica
print("d/dx (x^(1/2)) =", sp.diff(x**(1/2), x)) # Expoente fracionário
print("d/dx (sin(x)) =", sp.diff(sp.sin(x), x)) # Seno
print("d/dx (cos(x)) =", sp.diff(sp.cos(x), x)) # Cosseno
```

```
print("d/dx (tan(x)) =", sp.diff(sp.tan(x), x)) # Tangente
print("d/dx (e^x) =", sp.diff(sp.exp(x), x))    # Exponencial
print("d/dx (ln(x)) =", sp.diff(sp.ln(x), x))    # Logaritmo
print("d/dx (a^x) =", sp.diff(sp.Pow(2, x), x)) # e.g. a=2
print("d/dx (log_a(x)) =", sp.diff(sp.log(x, 2), x)) # e.g. a=2
```

Código-fonte 9 – Limites de funções básicas com SymPy
Fonte: Elaborado pelo autor (2025)

Já na seguinte imagem há propriedades de derivadas e o código-fonte “Propriedades de derivadas com SymPy” traz exemplos de implementações também com SymPy.

Propriedade	Descrição matemática	Resultado da derivada
Constante	$f(x) = c$	$f'(x) = 0$
Identidade	$f(x) = x$	$f'(x) = 1$
Potência	$f(x) = x^n$	$f'(x) = nx^{n-1}$
Constante multiplicativa	$f(x) = c \cdot g(x)$	$f'(x) = c \cdot g'(x)$
Soma	$f(x) = g(x) + h(x)$	$f'(x) = g'(x) + h'(x)$
Diferença	$f(x) = g(x) - h(x)$	$f'(x) = g'(x) - h'(x)$
Produto	$f(x) = g(x) \cdot h(x)$	$f'(x) = g'(x)h(x) + g(x)h'(x)$
Quociente	$f(x) = \frac{g(x)}{h(x)}$	$f'(x) = \frac{g'(x)h(x) - g(x)h'(x)}{h(x)^2}$
Regra da cadeia	$f(x) = g(h(x))$	$f'(x) = g'(h(x)) \cdot h'(x)$

Figura 10 – Exemplos de propriedades de derivadas
Fonte: Elaborada pelo autor (2025)

```
import sympy as sp
x = sp.Symbol('x')
# Regra da soma
print("d/dx (f(x) + g(x)) =", sp.diff(sp.sin(x) + sp.exp(x), x))
# Regra da multiplicação por uma constante
print("d/dx (c·f(x)) =", sp.diff(3 * x**2, x))
# Regra do produto
print("d/dx (f(x)·g(x)) =", sp.diff(sp.sin(x) * sp.exp(x), x))
# Regra do quociente
print("d/dx (f(x)/g(x)) =", sp.diff(sp.sin(x) / sp.exp(x), x))
# Regra da cadeia
print("d/dx (f(g(x))) =", sp.diff(sp.sin(x**2), x))
```

Código-fonte 10 – Propriedades de derivadas com SymPy
Fonte: Elaborado pelo autor (2025)

Agora, vamos fazer dois exemplos de derivadas. Embora o procedimento que será mostrado seja específico para uma função, ele ilustra o fluxo de trabalho fundamental do cálculo diferencial. Além disso, o processo mostra como podemos identificar as características de cada termo, consultar as derivadas fundamentais em tabelas de referência e aplicar as propriedades algébricas necessárias. Ao final, o objetivo é quebrar uma expressão aparentemente complexa em partes com soluções conhecidas para, assim, obter a derivada final.

O primeiro exemplo é uma função cúbica: $f(x)=x^3-3x^2+2$.

1. Inicialmente, identificamos que a função é a soma de três funções. Isso significa que podemos calcular cada derivada individualmente e depois somar os resultados.
2. Aplicando as regras de derivação termo a termo obtemos:
 - A derivada de x^3 é $3x^2$. Observe como a potência “caiu” como uma constante multiplicativa. Por esse motivo, ela se chama **regra do tombo**;
 - A derivada de $-3x^2$ é $-6x$, pela mesma regra do tombo;
 - A derivada de qualquer constante é zero.
3. Portanto, $f'(x)=3x^2-6x$, que descreve uma parábola.

O segundo exemplo é $f(x) = (3x^2 + 1) \ln(x)$.

1. Inicialmente, identificamos que a função é um produto de duas funções: $u(x) = 3x^2 + 1$ e $v(x) = \ln(x)$.
2. Retomamos a regra do produto: se $f(x) = u(x) \cdot v(x)$, então $f'(x) = u'(x) \cdot v(x) + u(x) \cdot v'(x)$.
3. Derivamos cada parte separadamente: $u'(x) = d/dx (3x^2 + 1) = 6x$
 $v'(x) = d/dx (\ln(x)) = 1/x$.
4. Substituímos na regra do produto: $f'(x) = 6x \cdot \ln(x) + (3x^2 + 1) \cdot (1/x)$.
5. Simplificamos o segundo termo: $(3x^2 + 1) / x = 3x + 1/x$.
6. Resultado final da derivada: $f'(x) = 6x \ln(x) + 3x + 1/x$.

A figura “Função $f(x) = (3x^2 + 1) \ln(x)$ e sua derivada” apresenta o comportamento de $f(x) = (3x^2 + 1) \ln(x)$ em conjunto com a sua respectiva derivada no mesmo plano cartesiano. Ambas as funções estão definidas estritamente para $x > 0$, respeitando o domínio do logaritmo natural. À medida que x aumenta, podemos notar visualmente que a inclinação de $f(x)$ torna-se positiva e crescente, o que justifica por que o gráfico da derivada permanece acima do eixo horizontal e em constante ascensão.

Sobre o comportamento assintótico próximo à origem, enquanto a função original tende ao infinito negativo devido à característica do $\ln(x)$, a sua derivada tende ao infinito positivo. Isso ocorre porque, conforme x se aproxima de zero pela direita, a curva original torna-se extremamente íngreme, o que se reflete em valores de derivada cada vez mais altos devido ao termo $1/x$ no cálculo analítico.

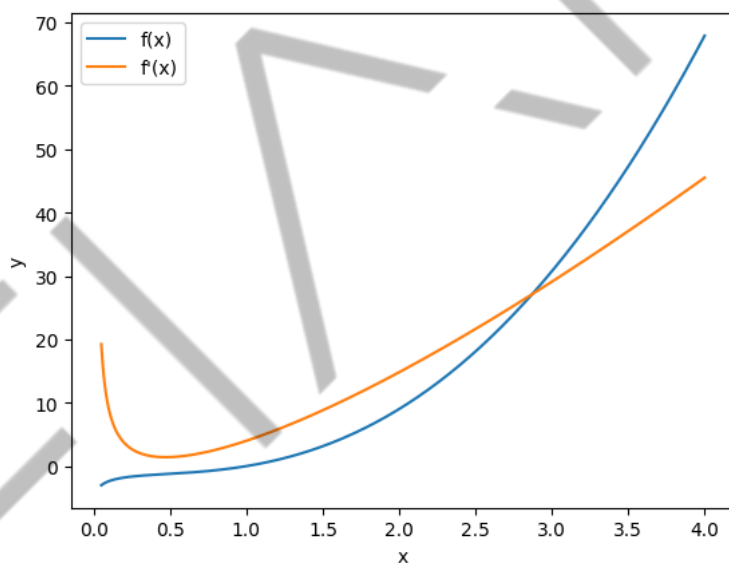


Figura 11 - Função $f(x) = (3x^2 + 1) \ln(x)$ e sua derivada
Fonte: Elaborada pelo autor (2025)

3.3 Funções que não são diferenciáveis em todos os pontos

Funções podem ser contínuas e ainda assim não serem diferenciáveis em todos os pontos. Um exemplo clássico é a função valor absoluto: $u(t) = |t|$. Ela é contínua em todos os números reais, mas não é diferenciável em $(t=0)$, pois o coeficiente angular à esquerda é (-1) e à direita é $(+1)$. Não existe, portanto, uma inclinação única nesse ponto.

No código-fonte abaixo, definimos $u(t)=|t|$ e usamos `np.gradient` para calcular uma derivada numérica. Esse método aproxima a derivada por diferenças finitas, funcionando bem onde a função é suave. Porém, próximo de $(t=0)$, a derivada numérica produz uma transição abrupta, na forma de uma reta vertical.

```
import numpy as np
# Eixo do tempo
t = np.linspace(-1, 1, 1000)
# Função valor absoluto
u = np.abs(t)
# Derivada numérica
du_dt = np.gradient(u, t)
```

Código-fonte 11 – Derivada numérica implementada com `numpy.gradient`
Fonte: Elaborado pelo autor (2025)

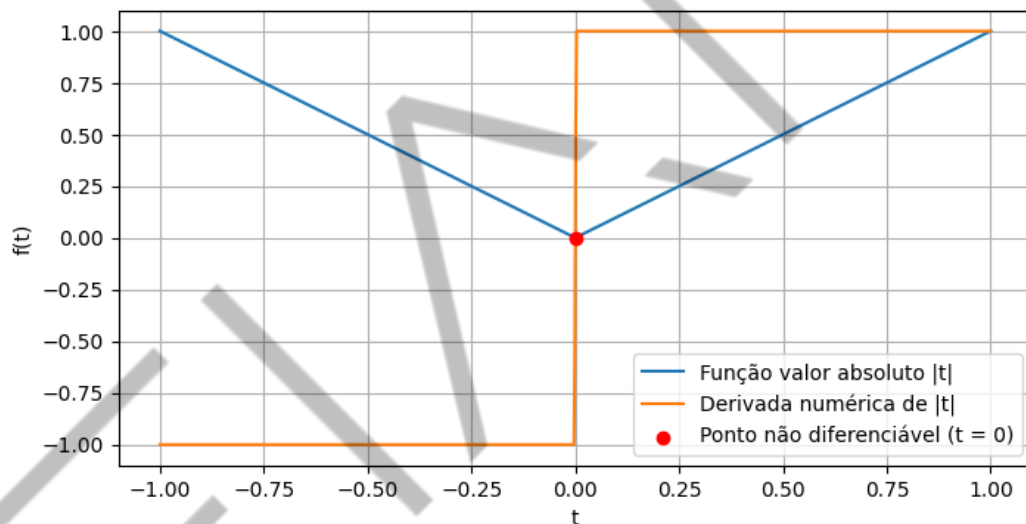


Figura 12 - Função valor absoluto e sua derivada (degrau unitário)
Fonte: Elaborada pelo autor (2025)

3.4 Derivadas de ordem superior

Depois de compreender a primeira derivada, que indica a taxa instantânea de variação de uma função e permite identificar onde ela cresce ou decresce, o estudo avança naturalmente para a aplicação em sequência de derivadas.

A segunda derivada descreve a variação da variação, e assim por diante. A figura “Diferentes notações para derivadas” mostra diferentes notações possíveis para elas, enquanto a imagem “Dois exemplos de quartas derivadas de funções” mostra dois exemplos de repetidas diferenciações de um polinômio e de um seno.

Tipo	1ª Derivada	2ª Derivada	3ª Derivada
Leibniz	$\frac{dy}{dx}$	$\frac{d^2y}{dx^2}$	$\frac{d^3y}{dx^3}$
Lagrange	$f'(x)$	$f''(x)$	$f'''(x)$
Newton	$\dot{x}$	$\ddot{x}$	$\dddot{x}$
Operador	Df	D^2f	D^3f

Figura 13 – Diferentes notações para derivadas
Fonte: Elaborada pelo autor (2025)

Ordem da Derivada	$f(x) = x^4 + 3x^3 - 0.5x^2 - 2.5x + 10$	$g(x) = \sin(x)$
$f(x)$	$x^4 + 3x^3 - 0.5x^2 - 2.5x + 10$	$\sin(x)$
$f'(x)$	$4x^3 + 9x^2 - x - 2.5$	$\cos(x)$
$f''(x)$	$12x^2 + 18x - 1$	$-\sin(x)$
$f'''(x)$	$24x + 18$	$-\cos(x)$
$f^{(4)}(x)$	24	$\sin(x)$

Figura 14 – Dois exemplos de quartas derivadas de funções
Fonte: Elaborada pelo autor (2025)

Para utilizar `Numpy` para obter derivadas de ordem superior, o procedimento é o mesmo, conforme código-fonte “Derivadas de ordem superior implementadas com `numpy.gradient`”. O desafio é que a aplicação repetida de derivadas numéricas tende a amplificar erros e instabilidades, principalmente nas regiões perto das extremidades. Na figura “Derivadas numéricas de ordem superior”, por exemplo, que mostra derivadas numéricas, foram omitidos os dois primeiros e últimos pontos de $f'(x)$, que apresentavam comportamentos instáveis perto das bordas. Em contrapartida, o uso de computação simbólica no código-fonte “Derivadas de ordem superior implementadas com `SymPy`” retorna resultados exatos ao usar o `sp.diff`, mas que poderiam ter algum erro numérico ao usar `lambdify` para cálculo de valores numéricos.

```
import numpy as np
def f(x):
    return x**4 - 3*x**3 + 2*x**2 - x + 1
x = np.linspace(-2, 3, 600)
dx = x[1] - x[0]
```

```

y = f(x)
dy_dx = np.gradient(y, dx)
d2y_dx2 = np.gradient(dy_dx, dx)
d3y_dx3 = np.gradient(d2y_dx2, dx)
d4y_dx4 = np.gradient(d3y_dx3, dx)

```

Código-fonte 12 - Derivadas de ordem superior implementadas com numpy.gradient.

Fonte: Elaborado pelo autor (2025)

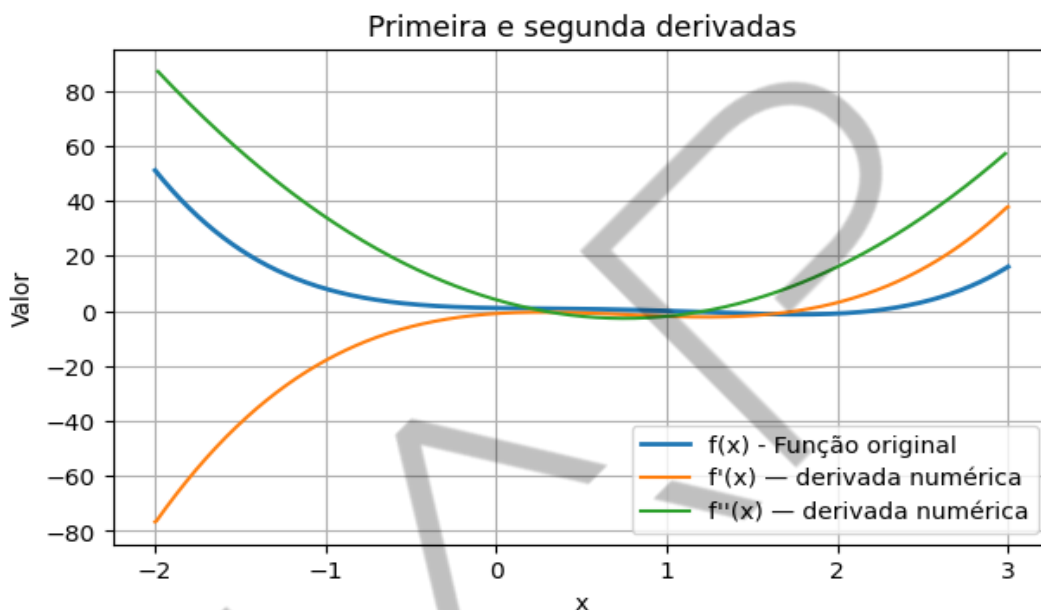


Figura 15 - Derivadas numéricas de ordem superior

Fonte: Elaborada pelo autor (2025)

```

import sympy as sp
import numpy as np

x = sp.symbols('x')
f = x**4 - 3*x**3 + 2*x**2 - x + 1

f1 = sp.diff(f, x)
f2 = sp.diff(f, x, 2)
f3 = sp.diff(f, x, 3)
f4 = sp.diff(f, x, 4)

f1_num = sp.lambdify(x, f1, 'numpy')
f2_num = sp.lambdify(x, f2, 'numpy')
f3_num = sp.lambdify(x, f3, 'numpy')
f4_num = sp.lambdify(x, f4, 'numpy')

x0 = 1.0
f1_num(x0), f2_num(x0), f3_num(x0), f4_num(x0)

```

Código-fonte 13 – Derivadas de ordem superior implementadas com SymPy

Fonte: Elaborado pelo autor (2025)

3.5 Pontos críticos

Um ponto crítico é um valor no domínio de uma função onde sua derivada é igual a zero ou não existe, representando momentos em que a função para de crescer ou decrescer para mudar de comportamento ou estabilizar.

O código-fonte “Código para localizar pontos críticos de uma função” identifica pontos críticos. Para isso, ele localiza onde a derivada muda de sinal ao usar `np.sign` para obter o sinal da derivada, `np.diff` para detectar mudanças desse sinal entre pontos consecutivos e `np.where` para retornar os índices onde essas mudanças ocorrem.

A figura “Pontos críticos de uma função polinomial” mostra o resultado desse código na região oscilatória de uma função polinomial. Observamos várias mudanças de direção no intervalo escolhido.

```
import numpy as np
a = -0.2
f = lambda x: x * (x - 1) * (x + 1) * (x - 2) * (x + 2) * (x + 1.5) * (x - 1.5)
df = lambda x: (f(x + 1e-6) - f(x - 1e-6)) / 2e-6

# Amostras
x_vals = np.linspace(-2.1, 2.1, 1000)
df_vals = df(x_vals)

zeros_idx = np.where(np.diff(np.sign(df_vals)))[0]
guesses = x_vals[zeros_idx]
crit_points = np.array(sorted(guesses))
```

Código-fonte 14 – Código para localizar pontos críticos de uma função

Fonte: Elaborado pelo autor (2025)

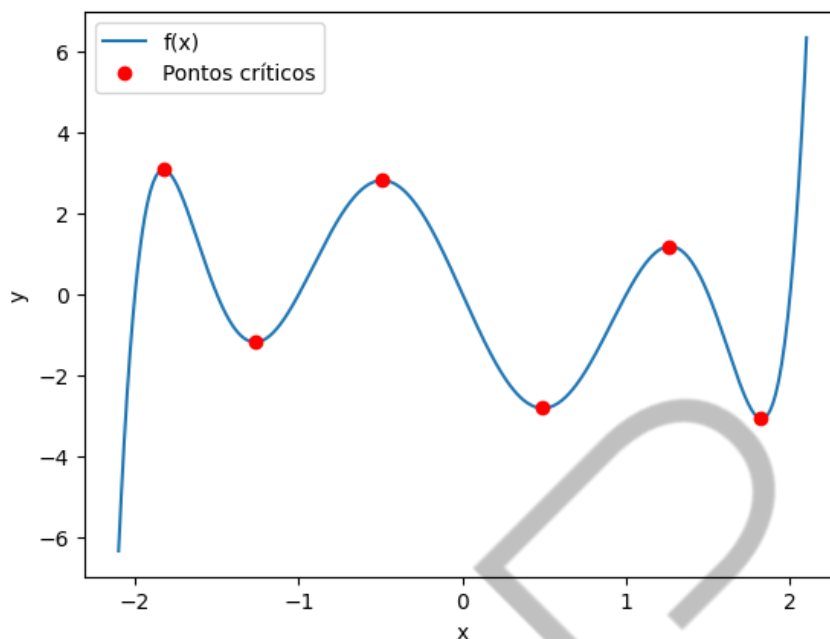


Figura 16 - Pontos críticos de uma função polinomial
Fonte: Elaborada pelo autor (2025)

No exemplo anterior, é relativamente direta a interpretação se esses pontos são de mínimos ou de máximos locais. No entanto, existem outras funções que exigem mais ferramentas para que consigamos obter informações a partir dos pontos críticos. Além de verificar se a primeira derivada se anula, podemos também avaliar o comportamento da segunda derivada, que, graficamente, está relacionada à concavidade da função. Ela permite saber se o gráfico está voltado para cima ou para baixo e em que regiões essa curvatura se altera.

Quando a primeira derivada se anula, a segunda ajuda a determinar se o ponto corresponde a um máximo local, um mínimo local ou apenas um ponto onde a função muda de comportamento sem atingir extremos. Além disso, a segunda derivada permite identificar pontos de inflexão, onde a concavidade da função muda, revelando transições importantes no formato do gráfico. Por vezes, esse tipo de análise é chamado de **teste da segunda derivada**.

A figura “**Informações que podem ser obtidas a partir da primeira e da segunda derivada**” mostra algumas informações que primeiras e segundas derivadas trazem ao analisar funções. Para facilitar o estudo e a aplicação prática, essa imagem também consolida os principais critérios utilizados para interpretar o comportamento de uma função. Ela relaciona as condições da primeira e da segunda derivada com

as conclusões geométricas e analíticas que podemos extrair sobre pontos críticos, extremos locais e a curvatura da função.

Condição	Conclusão
$f'(x) = 0$	x é um ponto crítico
$f'(x)$ não existe	x é um ponto crítico
$f'(x_0)$ muda de positivo para negativo	Máximo local em x_0
$f'(x_0)$ muda de negativo para positivo	Mínimo local em x_0
$f'(x_0)$ não muda de sinal	Não há extremo local
$f''(x) > 0$	Função côncava para cima
$f''(x) < 0$	Função côncava para baixo
$f''(x_0) = 0$	x_0 é candidato a ponto de inflexão
$f''(x)$ muda de sinal em x_0	Ponto de inflexão em x_0
$f''(x_0) = 0$ sem mudança de sinal	Não é ponto de inflexão
$f'(x_0) = 0$ e $f''(x_0) > 0$	Mínimo local
$f'(x_0) = 0$ e $f''(x_0) < 0$	Máximo local
$f'(x_0) = 0$ e $f''(x_0) = 0$	Teste inconclusivo

Figura 17 – Informações que podem ser obtidas a partir da primeira e da segunda derivada
Fonte: Elaborada pelo autor (2025)

Para ilustrar as nuances anteriores, vamos considerar a função $f(x) = x^3$, cujo comportamento está detalhado na figura “Avaliação da primeira e da segunda derivada de x^3 em $x=0$ ”. Através das regras de derivação, obtemos suas duas primeiras derivadas: $f'(x) = 3x^2$ e $f''(x) = 6x$.

No ponto $x_0 = 0$, $f'(0) = 0$ e $f''(0) = 0$. Nesse cenário, o teste da segunda derivada resulta em uma condição inconclusiva, não sendo capaz de determinar a natureza do ponto.

Entretanto, ao observarmos o comportamento da função ao redor da origem, percebemos que $f(x)$ permanece crescente tanto para valores negativos quanto para valores positivos de x . Isso significa que, embora a reta tangente fique momentaneamente horizontal em $x = 0$, a função não atinge um "pico" ou um "vale"; ela apenas suaviza sua inclinação para continuar subindo. O ponto $x = 0$ é classificado como um ponto de inflexão com tangente horizontal, marcando o local onde a concavidade da curva muda de "para baixo" até "para cima".

Esse exemplo mostra que $f'(x_0) = 0$ não garante a existência de um extremo local e que $f''(x_0) = 0$ impede a conclusão pelo teste da segunda derivada, sendo necessário aprofundar a análise para entender o comportamento ou explorar informações disponíveis como gráficos (em casos de muitas dimensões, essa não é uma opção viável).

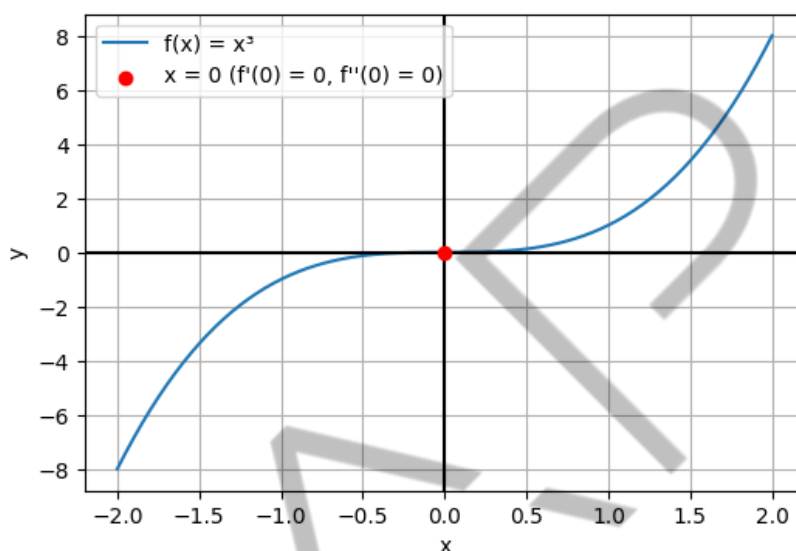


Figura 18 - Avaliação da primeira e da segunda derivada de x^3 em $x=0$
Fonte: Elaborada pelo autor (2025)

4 APLICAÇÕES DE LIMITES E DERIVADAS

Com base nos conceitos discutidos, vamos entender abordagens numéricas e simbólicas em aplicações que envolvem limites e derivadas.

4.1 Juros compostos e a constante de Euler com limites

Juros são a remuneração do dinheiro investido ou a taxa cobrada pelo uso de um dinheiro emprestado. Em qualquer uma dessas situações, o objetivo é relacionar três quantidades fundamentais: o capital inicial (C), a taxa de juros (r) e o tempo (n ou t). A partir delas, queremos calcular o montante (M), isto é, o valor final após a incidência dos juros.

Nos juros simples, o valor dos juros é sempre calculado sobre o capital inicial, de acordo com:

$$J_s = C \cdot r \cdot n$$

Em que **C** é o capital inicial, **r** é a taxa de juros (em forma decimal) e **n** é o número de períodos. O montante final é obtido somando os juros ao capital inicial:

$$M = C + Js$$

Substituindo J_s pela expressão anterior, obtemos:

$$M = C + C \cdot r \cdot n$$

$$M = C \cdot (1 + r \cdot n)$$

Dessa forma, o valor acrescentado a cada período é sempre o mesmo.

Nos juros compostos, os juros passam a incidir não apenas sobre o capital inicial, mas também sobre os juros acumulados nos períodos anteriores. Por isso dizemos que há “juros sobre juros”.

Considere um empréstimo de 1000 reais a uma taxa de 10% ao ano.

- Após o primeiro ano, o montante é: $M_1 = 1000 \cdot (1 + 0,1) = 1000 \cdot 1,1 = 1100$.
- Após o segundo ano: $M_2 = 1000 \cdot (1,1 \cdot 1,1) = 1000 \cdot (1,1)^2$.
- Após o terceiro ano: $M_3 = 1000 \cdot (1,1 \cdot 1,1 \cdot 1,1) = 1000 \cdot (1,1)^3$.

De forma geral, o montante em juros compostos é dado por:

$$M = C \cdot (1 + r)^t$$

Em que **t** representa o número de períodos considerados.

Essa fórmula pode ser generalizada quando a incidência de juros ocorre em intervalos menores do que o período total. Mantendo a taxa **r** associada ao período **t** (por exemplo, taxa anual) e dividindo esse período em **n** partes iguais, a taxa por subperíodo passa a ser **r / n**, e o número total de incidências passa a ser **t · n**.

Nesse caso, o montante é dado por:

$$M = C \cdot (1 + r / n)^{(t \cdot n)}$$

Em que:

- **C** é o capital inicial;
- **r** é a taxa de juros do período **t**;
- **t** é o tempo total considerado;

- **n** é o número de vezes por período em que os juros são compostos.
Exemplos:

- $n = 1 \rightarrow$ capitalização anual;
- $n = 12 \rightarrow$ capitalização mensal;
- $n = 52 \rightarrow$ capitalização semanal;
- $n = 365 \rightarrow$ capitalização diária.

À medida que o valor de **n** aumenta, a capitalização se torna mais frequente e o montante final cresce. Surge então a questão: esse crescimento continua indefinidamente ou se aproxima de um valor limite? Para responder esse questionamento, vamos considerar um caso particular:

- Capital inicial $C = 1$;
- Taxa de juros $r = 1$ (ou seja, 100%);
- Tempo $t = 1$ ano.

Substituindo esses valores na expressão do montante, obtemos:

$$M(n) = (1 + 1/n)^n$$

Quando **n** assume valores cada vez maiores, essa expressão se aproxima de um valor fixo. Esse valor é o número de Euler, denotado por **e**, cujo valor aproximado é: $e \approx 2,71828$.

Matematicamente, isso corresponde ao limite:

$$\lim_{x \rightarrow \infty} \left(1 + \frac{1}{x}\right)^x = e$$

Figura 19 – Limite com o número de Euler
Fonte: Elaborada pelo autor (2026)

Esse é exatamente o limite calculado no código em Python com a biblioteca `SymPy` no código-fonte a seguir, mostrando que o número **e** surge naturalmente como o valor limite do montante em um regime de juros compostos com capitalização instantânea.

```
from sympy import *
```

```
x= symbols('x')
r = limit((1+1/x)**x, x, oo)
print(r)
```

Código-fonte 15 – Código de geração do limite
Fonte: Elaborado pelo autor (2025)

4.2 Diferenciação simbólica com SymPy com aplicação na física

Anteriormente, discutimos sobre como as funções de posição $x(t)$, velocidade $v(t)$ e aceleração $a(t)$ estão relacionadas. Em problemas de movimento uniformemente variado, considera-se um polinômio de segundo grau para descrever a posição:

$$x(t) = x_0 + v_0 t + (1/2)a_0 t^2$$

Nessa operação, o subscrito 0 indica que são grandezas iniciais. Vamos avaliar o porquê.

A derivada de $x(t)$ descreve a velocidade $v(t) = a_0 t + v_0$ (crescimento linear). Por sua vez, a derivada de $v(t)$ descreve a aceleração $a(t) = a_0$ (constante). Em outras palavras, o polinômio de segundo grau é a única opção plausível quando a aceleração é constante e a velocidade cresce linearmente.

Para verificar essas relações, o código-fonte abaixo traz a diferenciação simbólica das expressões. A figura “Relação entre posição, velocidade e aceleração em um movimento uniformemente variado” ilustra as três funções juntas quando $x_0 = 0$ [m], $v_0 = 0$ [m/s] e $a_0 = 2$ [m/s²], em um intervalo de 60 segundos.

```
import numpy as np
import sympy as sp
t = sp.symbols('t')
x0, v0, a = sp.symbols('x0 v0 a')
# posição, velocidade e aceleração no MUV
x_t = x0 + v0*t + sp.Rational(1, 2)*a*t**2
v_t = sp.diff(x_t, t)
a_t = sp.diff(x_t, t, 2)
# valores iniciais
x0_val, v0_val, a_val = 0.0, 0.0, 2.0
t_vals = np.linspace(0, 60, 300)
# funções numéricas
x_num = sp.lambdify(t, x_t.subs({x0: x0_val, v0: v0_val, a: a_val}),
'numpy')
v_num = sp.lambdify(t, v_t.subs({v0: v0_val, a: a_val}), 'numpy')
```

```
a_vals = a_val * np.ones_like(t_vals)
```

Código-fonte 16 – Conversão de variáveis entre SymPy e NumPy
Fonte: Elaborado pelo autor (2025)

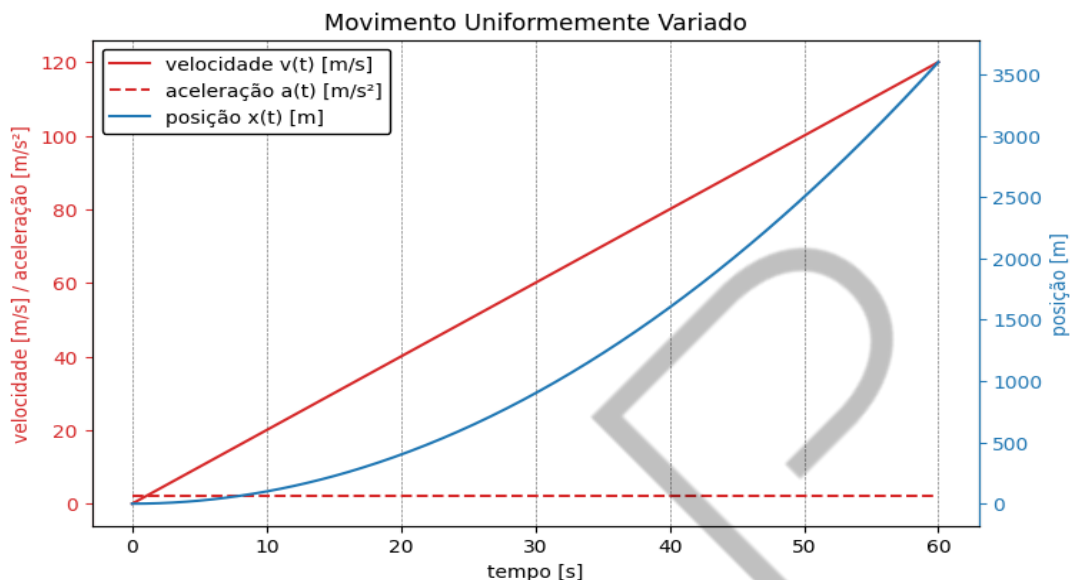


Figura 20 - Relação entre posição, velocidade e aceleração em um movimento uniformemente variado

Fonte: Elaborada pelo autor (2025)

4.3 Regra de L'Hôpital

O uso de derivadas pode facilitar a solução de limites na forma indeterminada, como $0/0$ ou ∞/∞ , através da regra de L'Hôpital.

Por exemplo, seja $f(x) = \sin(x)/x$. Para calcular o limite de $f(x)$ quando $x \rightarrow 0$, essa regra nos informa que o resultado desse limite será o mesmo que se derivarmos numerador e denominador e depois aplicarmos o mesmo limite. Após diferenciação, $f'(x) = \cos(x)$, cuja avaliação em $x=1$ resulta no limite igual a 1. Uma interpretação rápida desse resultado é que $\sin(x) \approx x$ para valores pequenos.

O código-fonte “Exemplo do uso da regra de L'Hôpital na solução de limites na forma indeterminada” traz esse resultado através do `SymPy`, enquanto a figura “Visualização do limite de $\sin(x)/x$ para $x \rightarrow 0$ e solução com L'Hôpital” mostra graficamente como as duas expressões se encontram em $x=0$.

Observação: a importância do limite de $\sin(x)/x$ com $x \rightarrow 0$ reside principalmente na demonstração da obtenção da derivada de $\sin(x)$. Aqui, apenas utilizamos esse resultado para ilustrar a regra de L'Hôpital.

```
import sympy as sp
import numpy as np
# Variável simbólica
x = sp.symbols('x')
# Funções
f = sp.sin(x) / x      # função original
g = sp.cos(x)         # após L'Hôpital
# Limites simbólicos
lim_f = sp.limit(f, x, 0)
lim_g = sp.limit(g, x, 0)
print("Limite de sin(x)/x quando x → 0:", lim_f)
print("Limite de cos(x) quando x → 0:", lim_g)
```

Código-fonte 17 – Exemplo do uso da regra de L'Hôpital na solução de limites na forma indeterminada

Fonte: Elaborado pelo autor (2025)

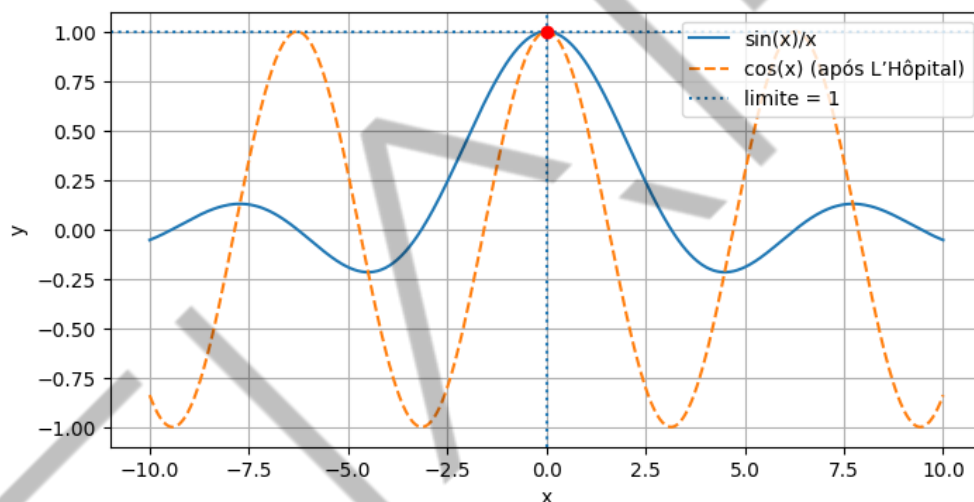


Figura 21 - Visualização do limite de $\sin(x)/x$ para $x \rightarrow 0$ e solução com L'Hôpital

Fonte: Elaborada pelo autor (2025)

4.4 Expansão de Taylor de primeira ordem

Quando queremos aproximar uma função não-linear por uma reta em torno de um ponto conhecido, podemos usar a ideia de expansão de Taylor de primeira ordem. Conhecendo o valor da função em um ponto x_0 e a sua derivada naquele ponto, podemos construir uma aproximação linear dessa função em um intervalo próximo de x_0 . Em notação matemática, esse processo de linearização é descrito através de:

$$f(x) \approx f(x_0) + f'(x_0)(x - x_0)$$

O código-fonte “Aproximação de Taylor de primeira ordem usando expressões analíticas” mostra o exemplo da função raiz quadrada. Nesse caso, utilizou-se a derivada exata dessa função, já que ela é conhecida. O ponto escolhido foi $x=9$, no

intervalo de $8 < x < 10$. Observamos na figura “Linearização da função raiz quadrada de x com Taylor de 1ª ordem” que a aproximação linear é eficiente para a região escolhida.

A importância desse procedimento reside no fato de que é muito mais fácil lidar com modelos lineares do que com modelos não-lineares. Na medida que a nossa aproximação linear é razoável para representar o fenômeno, isso simplifica muito a sua análise. Um exemplo da expansão de Taylor é no movimento do pêndulo simples, em que a função $\sin(\theta)$ é aproximada para θ quando se considera apenas ângulos pequenos para o seu movimento.

```
import numpy as np
# Função e derivada
f = lambda x: np.sqrt(x)
df = lambda x: 1 / (2 * np.sqrt(x))
a = 9 # Ponto de aproximação
x_full = np.linspace(0, 16, 400) # Domínio global
y_full = f(x_full)
x_local = np.linspace(a - 1, a + 1, 200) # Domínio local
y_taylor = f(a) + df(a) * (x_local - a)
```

Código-fonte 18 – Aproximação de Taylor de primeira ordem usando expressões analíticas
Fonte: Elaborado pelo autor (2025)

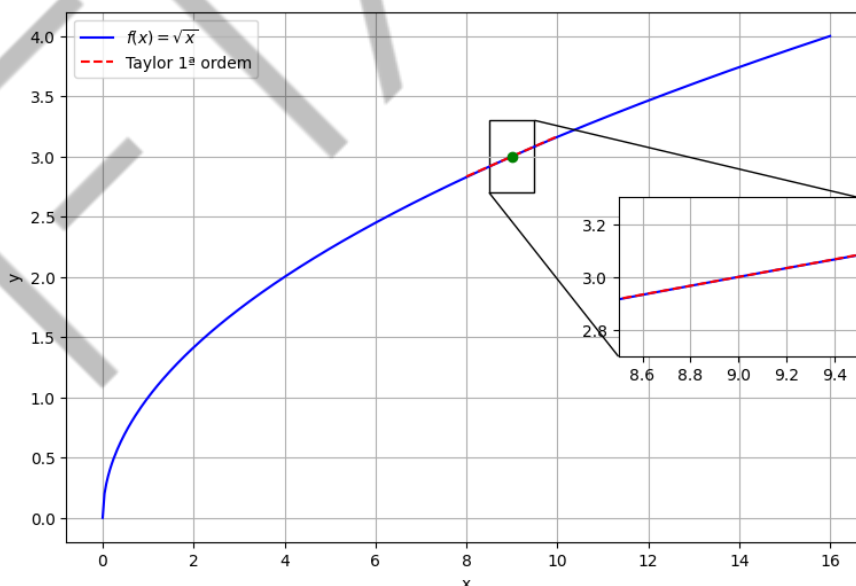


Figura 22 - Linearização da função raiz quadrada de x com Taylor de 1ª ordem
Fonte: Elaborada pelo autor (2025)

4.5 Otimização com gradiente descendente: Casos 2D e 3D

Compreender a derivada como essa inclinação instantânea permite aplicá-la na resolução de problemas de otimização, que são aqueles que maximizam ou minimizam funções. Se buscamos o ponto máximo de uma função e a derivada é positiva, sabemos que devemos continuar avançando no valor de x para aumentar o valor da função. A mesma coisa poderia ser dita de minimizações, em que olhamos para a direção de derivada e escolhemos o caminho que devemos seguir na função.

Vamos considerar uma situação em que $f(x)$ é conhecida de forma explícita e representa uma quantidade que queremos minimizar, como um custo ou gasto. Existem diversos algoritmos para minimização e entre eles está o gradiente descendente. Em cada ponto do domínio, o sinal da derivada $f'(x)$ da função indica se a função $f(x)$ cresce (sinal positivo) ou diminui (sinal negativo). Assim, define-se uma atualização da grandeza de interesse no sentido em que ela sempre diminui, isto é, na direção oposta da derivada.

O algoritmo é iterativo e repete esse processo pelo número de vezes desejada: escolhe um ponto inicial, avalia a variação local da função nesse ponto e dá um pequeno passo na direção contrária ao crescimento. Repetindo esse procedimento, o ponto percorre a superfície da função, idealmente descendo progressivamente em direção a um mínimo.

No caso 1D, isso pode ser escrito como:

$$x^{t+1} = x^t - \eta f'(x^t)$$

Nessa operação, η é o tamanho do passo, também conhecido como taxa de aprendizagem, $f'(x)$ é a derivada avaliada no ponto atual e o índice t indica a iteração do algoritmo.

Não é necessário conhecer toda a função de uma vez. Utilizando apenas informações locais no ponto atual, é possível calcular a próxima iteração. Cada passo ajusta a posição no espaço, aproximando-se de regiões onde a função assume valores menores. A rapidez, a estabilidade e o caminho percorrido pelo algoritmo dependem da forma da função, do ponto inicial e do tamanho do passo escolhido.

O código-fonte “Implementação de gradiente descendente 1D com derivada numérica” traz a implementação aplicada na função $x^2 \sin(x)$, enquanto a figura

“Otimização com gradiente descendente obtendo um mínimo local” mostra o gráfico da função, o ponto inicial da otimização e o caminho até ela alcançar um mínimo. Nota-se que a solução final foi de um mínimo local, por conta do ponto inicial utilizado.

```
import numpy as np
# função e parâmetros
f = lambda x: x**2 * np.sin(x)
x0 = 8.0
alpha = 0.01
n_iter = 15
eps = 1e-5
# gradiente numérico (1D)
grad = lambda x: (f(x + eps) - f(x - eps)) / (2 * eps)
# gradiente descendente
x = x0
hist = [x]
for _ in range(n_iter):
    x = x - alpha * grad(x)
    hist.append(x)
hist = np.array(hist)
```

Código-fonte 19 – Implementação de gradiente descendente 1D com derivada numérica
Fonte: Elaborado pelo autor (2025)

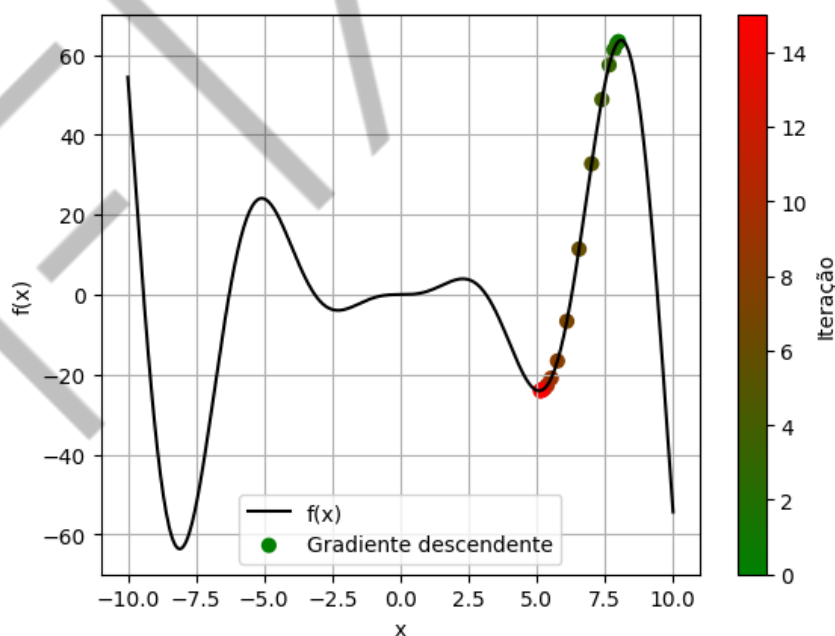


Figura 23 - Otimização com gradiente descendente obtendo um mínimo local
Fonte: Elaborada pelo autor (2025)

Em problemas mais gerais, a função pode depender de várias variáveis ao mesmo tempo. Quando a função depende de uma única variável, é possível visualizar

o problema como um gráfico em uma dimensão: basta olhar para a curva e identificar seus vales. Em duas ou três dimensões, porém, a situação muda. A função define uma superfície ou um relevo, e o ponto de menor valor pode estar escondido em uma região difícil de localizar apenas por inspeção. É nesse contexto que entram os métodos numéricos: procedimentos algorítmicos que exploram a função passo a passo, usando apenas informações locais.

Nesses casos, não existe uma única derivada, mas uma derivada em relação a cada variável. Desse conceito que vem o nome gradiente, que é apenas a união dessas derivadas, indicando a direção em que a função como um todo cresce mais rapidamente. No caso particular de uma única variável, o gradiente coincide com a própria derivada. Agora, a relação é dada por:

$$x^{(t+1)} = x^t - \eta \nabla f(x^t)$$

Aqui, x é um vetor e ∇ representa o operador gradiente.

Em 3D, a lógica continua a mesma: cada iteração ajusta a posição no espaço com base nas informações locais da função, aproximando-se progressivamente de regiões onde a função assume valores menores. Assim, vamos ilustrar esse caso com a Himmelblau, uma função de duas variáveis reais. Se representarmos o vetor de entrada por $x = (x_1, x_2)$, ela é definida por:

$$f(x_1, x_2) = (x_1^2 + x_2 - 11)^2 + (x_1 + x_2^2 - 7)^2$$

Seja o caso em que o ponto inicial é $x_0 = (-10, 10)$. A figura “Gráfico de contornos (em cima) e gráfico da superfície durante minimização com gradiente descendente (embaixo)” mostra a forma dessa função em 2D (contornos) e 3D (superfície), bem como os valores de x ao longo das iterações. Observa-se que, se a função for diferenciável (que neste caso é), o gradiente é perpendicular (normal) às curvas de contorno. Assim, rapidamente ela caminha para o mínimo da função.

Seja para essa função ou outras, caso escolhêssemos um valor menor de η , mais iterações seriam necessárias para chegar em um mínimo, seja ele local ou global. Caso o valor de η fosse maior, por vezes ele poderia chegar no mesmo mínimo, mas saltos grandes poderiam levar a solução para outras regiões, dependendo se a função apresenta muitos mínimos locais.

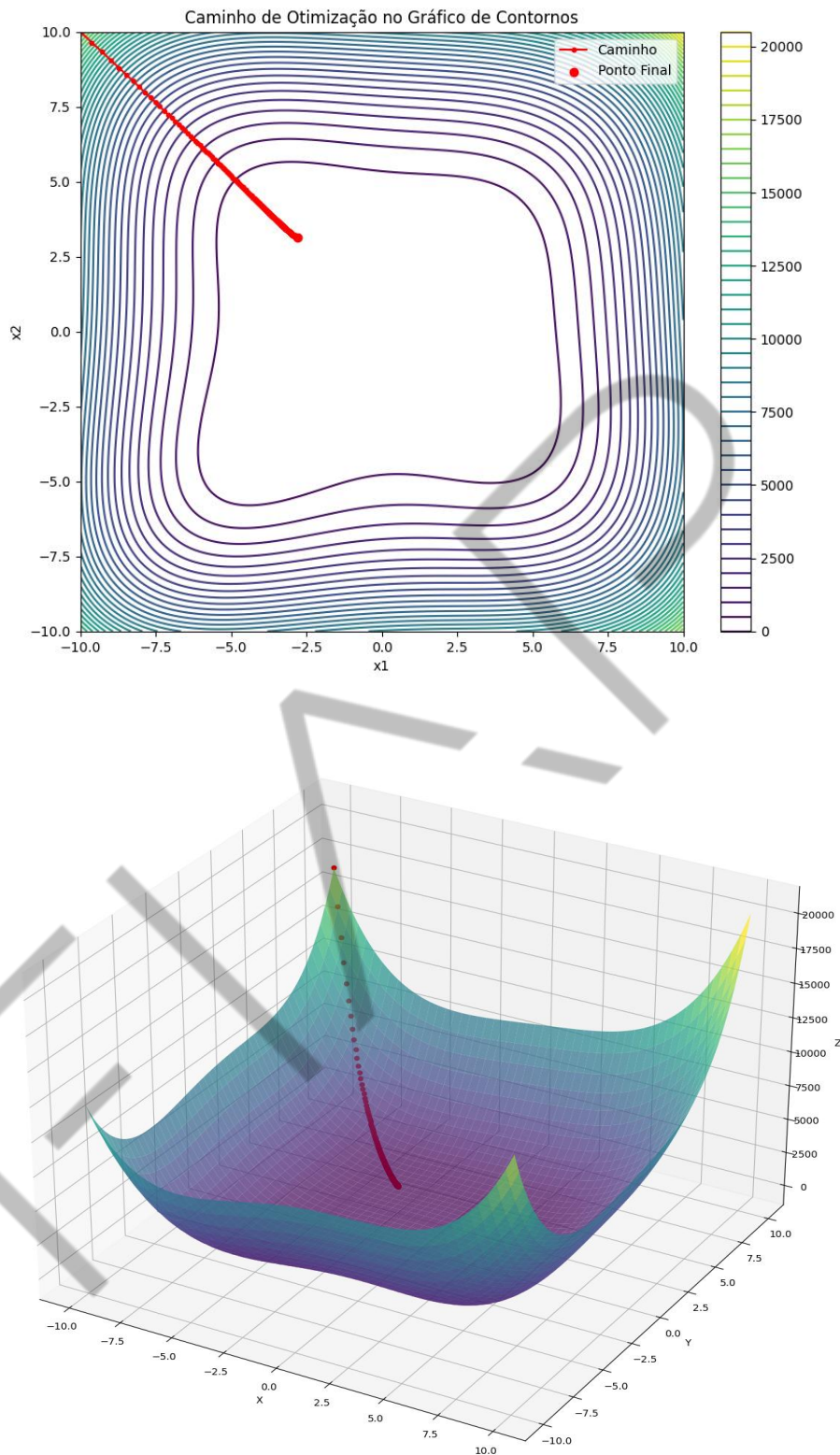


Figura 24 - Gráfico de contornos (em cima) e gráfico da superfície durante minimização com gradiente descendente (embaixo)

Fonte: Elaborada pelo autor (2025)

4.6 Otimização com SciPy

Esse exemplo é bastante semelhante com o problema de otimização visto anteriormente. A diferença é que, ao invés de escrever explicitamente o algoritmo de otimização, vamos utilizar a biblioteca `SciPy`, que também oferece ferramentas prontas para encontrar mínimos de funções de forma rápida e eficiente.

A função `scipy.optimize.minimize` permite fornecer uma função objetivo e retorna um ponto em que a função atinge um valor mínimo local. Em outras palavras, é possível minimizar funções de várias variáveis sem precisar implementar manualmente o passo a passo do algoritmo, além de possibilitar a escolha entre diferentes otimizadores, como gradiente descendente, BFGS ou método de Newton.

O código-fonte “Minimização de funções com `scipy.optimize.minimize`” mostra um exemplo em que escolhemos dois pontos iniciais diferentes para a minimização. A figura “Minimização de função com SciPy” mostra o resultado da otimização, em que pontos de cor igual correspondem ao início (pico) e fim (vale) da otimização.

Observamos que, apesar dos dois pontos iniciais serem muito próximos, o resultado é bem diferente. Enquanto um caiu para o mínimo local da esquerda, o outro caiu para o da direita. Esse exemplo também mostra como métodos determinísticos e baseados em gradientes podem levar a resultados substancialmente diferentes apenas com pequenas variações nos argumentos de entrada, como valores iniciais e tamanho do passo, indicando que não é trivial minimizar funções com diferentes mínimos locais.

```
import numpy as np
from scipy import optimize
def f(x):
    return 16*x**5 - 20*x**3 + 5*x
x = np.linspace(-1, 1, 500)
# Pontos iniciais
x0_list = [0.25, 0.35]
for x0 in x0_list:
    res = optimize.minimize(f, x0=x0, method="L-BFGS-B")
    print(res.x)
```

Código-fonte 20 – Minimização de funções com `scipy.optimize.minimize`
Fonte: Elaborado pelo autor (2025)

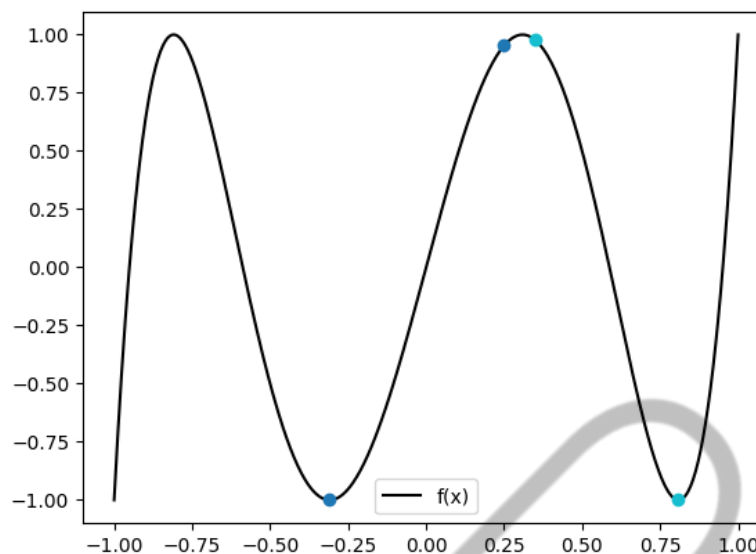


Figura 25 - Minimização de função com SciPy
Fonte: Elaborada pelo autor (2025)

4.7 Diferenciação automática com JAX e ajuste de curvas

Diferenciação automática (*autodiff*) engloba as técnicas para obter derivadas em relação à todas as variáveis de funções especificadas por programas de computador. É uma abordagem que não é nem numérica nem simbólica. Ela vai armazenando a forma como funções básicas se concatenam em um grafo de computação e, aplicando a regra da cadeia de forma iterativa, obtém o valor exato da derivada sem a ineficiência da diferenciação simbólica ou o erro de arredondamento da numérica. É importante ressaltar que *autodiff* é a base da otimização, do aprendizado de máquina e da aprendizagem profunda, viabilizando o treinamento de modelos complexos.

Vamos ilustrar um exemplo utilizando JAX, que é uma biblioteca que possui recursos de *autodiff*. No código-fonte “[Uso de diferenciação automática para cálculo de gradientes](#)”, definimos uma função $f(x)$ e rapidamente obtemos o seu gradiente, que pode ser avaliado em pontos específicos. Já no código-fonte “[Comparação de resultados de diferenciação com abordagem numérica, simbólica e autodiff](#)”, é realizada uma comparação do resultado numérico entre as abordagens numérica, simbólica e através de diferenciação automática.

```
import jax.numpy as jnp
from jax import grad
def f(x):
    return x**3 + 2*x
```

```
df = grad(f) # Calcula o gradiente de f (neste caso 1D, derivada).
df(3.0) #Avalia o gradiente em um ponto
```

Código-fonte 21 – Uso de diferenciação automática para cálculo de gradientes
Fonte: Elaborado pelo autor (2025)

```
import sympy as sp
import numpy as np
from jax import grad

# simbólico
x = sp.symbols('x')
f_sym = sp.sin(x) * sp.exp(x)
df_sym = sp.diff(f_sym, x)

# numérico
f_num = lambda x: np.sin(x) * np.exp(x)
def derivada_centrada(f, x, h=1e-5):
    return (f(x+h) - f(x-h)) / (2*h)

# autodiff
import jax.numpy as jnp
def f_auto(x):
    return jnp.sin(x) * jnp.exp(x)
df_auto = grad(f_auto)
x0 = 1.0
df_sym.subs(x, x0), derivada_centrada(f_num, x0), df_auto(x0)
```

Código-fonte 22 – Comparação de resultados de diferenciação com abordagem numérica, simbólica e autodiff

Fonte: Elaborado pelo autor (2025)

Vamos realizar um ajuste de curvas em pontos experimentais com *autodiff*, ou seja, vamos modificar os parâmetros de uma função para que ela consiga representar da melhor maneira possível aqueles pontos. Para isso, foi implementado o método do gradiente descendente com JAX. Dois casos foram avaliados: o ajuste de uma reta e o de uma parábola. Observa-se que, por mais que a minimização seja eficiente, não vai existir uma função linear que passe por todos os pontos experimentais.

```
import jax.numpy as jnp
from jax import grad
import numpy as np
x = jnp.array([0., 1., 2., 3., 4.])
y = jnp.array([1., 2., 2.5, 4., 4.5])
def loss(params): # Função de custo
    a, b = params
    y_pred = a*x + b
    return jnp.mean((y_pred - y)**2)
grad_loss = grad(loss) # Gradiente
```

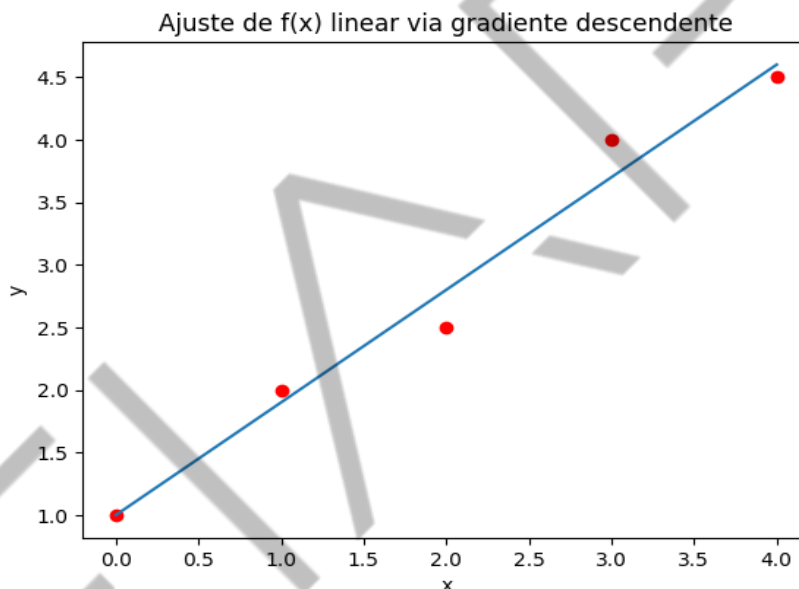
```

params = jnp.array([0.0, 0.0]) # Valores iniciais dos parâmetros
lr = 0.1 # Taxa de aprendizagem
for _ in range(1000):
    params = params - lr * grad_loss(params) # Gradiente descendente
a, b = params
# Converter para NumPy (Matplotlib não usa JAX diretamente)
x_np = np.array(x)
y_np = np.array(y)
# Reta ajustada
x_line = np.linspace(x_np.min(), x_np.max(), 100)
y_line = a * x_line + b

```

Código-fonte 23 – Ajuste de $f(x)$ linear via gradiente descendente

Fonte: Elaborado pelo autor (2025)

Figura 26 - Ajuste de $f(x)$ linear via gradiente descendente

Fonte: Elaborada pelo autor (2025)

```

import jax.numpy as jnp
from jax import grad
import numpy as np
x = jnp.array([0., 1., 2., 3., 4.])
y = jnp.array([0., 1., 4., 9., 16.])
def loss(params): # Função de custo
    a, b, c = params
    y_pred = a*x**2 + b*x + c
    return jnp.mean((y_pred - y)**2)
grad_loss = grad(loss) # Gradiente
params = jnp.array([0.0, 0.0, 0.0]) # Valores iniciais dos parâmetros
lr = 0.01 # Taxa de aprendizagem
for _ in range(100):
    params = params - lr * grad_loss(params) # Gradiente descendente

```

```
a, b, c = params
# Converter para NumPy (Matplotlib não usa JAX diretamente)
x_np = np.array(x)
y_np = np.array(y)
# Reta ajustada
x_line = np.linspace(x_np.min(), x_np.max(), 100)
y_line = a * x_line**2 + b * x_line + c
```

Código-fonte 24 – Ajuste de $f(x)$ na forma de parábola via gradiente descendente

Fonte: Elaborado pelo autor (2025)

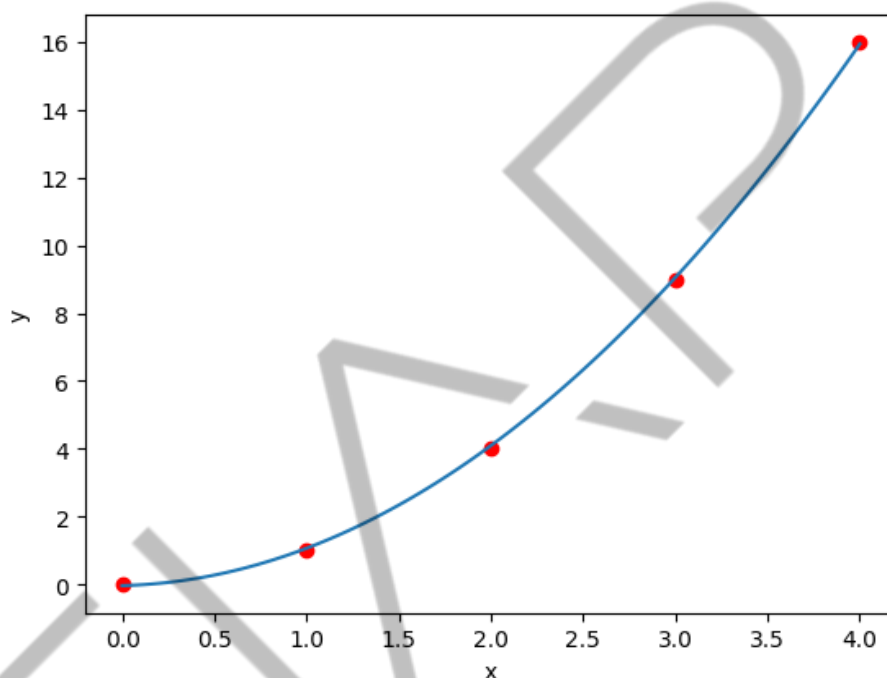


Figura 27 - Ajuste de $f(x)$ na forma de parábola via gradiente descendente

Fonte: Elaborada pelo autor (2025)

5 CONCLUSÃO

Anteriormente, o foco esteve na compreensão de funções enquanto objetos matemáticos, como regras que associam elementos de um conjunto a outro. Vimos diversos exemplos de funções descritas por expressões analíticas e suas características, usualmente interpretadas por meio de gráficos, iniciando o uso de ferramentas computacionais para implementá-las.

Nesse capítulo, passamos do estudo das funções em si para a análise de seus comportamentos, introduzindo conceitos e procedimentos que permitem investigar como uma função se comporta localmente e como ela responde a variações na

variável independente. Essa transição marca o início do cálculo diferencial e estabelece uma abordagem mais analítica do estudo das funções.

O primeiro conceito introduzido foi o de **limite**, que fornece uma linguagem matemática precisa para descrever o comportamento de uma função nas proximidades de um ponto. Sua importância reside na capacidade de analisar cenários em que não seria possível apenas substituindo valores na sua expressão original: pontos de descontinuidade, regiões de indefinição ou comportamentos singulares, como oscilações bruscas e crescimentos não limitados. O limite permite descrever tendência de uma função mesmo quando não podemos de fato utilizar aquele ponto específico.

A partir do limite, definimos a **derivada**. Ela funciona como um operador que gera uma nova função, capaz de indicar a taxa de variação instantânea da função original em qualquer ponto. Geometricamente, a derivada representa a inclinação da reta tangente ao gráfico. Na prática, esse conceito é o que nos permite identificar onde uma função cresce ou decresce, encontrar seus pontos críticos e determinar valores máximos e mínimos, o que é fundamental para resolver problemas de otimização e física.

No campo computacional, esses operadores são aplicados de forma prática. Ferramentas como a biblioteca `SymPy` permitem calcular limites e derivadas de forma simbólica, mantendo a precisão analítica. Já bibliotecas como JAX utilizam a diferenciação automática para processar derivadas de maneira eficiente em problemas complexos e com muitos dados. Assim, o cálculo diferencial se consolida como uma ferramenta indispensável para criar modelos matemáticos e algoritmos aplicados à tecnologia moderna, abrindo caminho para aplicações cada vez mais sofisticadas.

REFERÊNCIAS

DEISENROTH, Marc Peter; FAISAL, Aldo; ONG, Cheng Soon. **Mathematics for Machine Learning**. Cambridge: Cambridge University Press, 2020. ISBN 978-1108455145.

FARRELL, Peter; FUENTES, Alvaro; KOLHE, Ajinkya Sudhir; NGUYEN, Quan; SARVER, Alexander Joseph; TSATSOS, Marios. **The statistics and calculus with Python workshop**. Birmingham: Packt Publishing, 2020. ISBN 978-1800209763.

FERNANDES, Daniela Barude (org.). **Cálculo diferencial**. São Paulo, SP: Pearson Education do Brasil, 2014. 116 p., il. ISBN 9788543005423.

GONÇALVES, Mirian Buss; FLEMMING, Diva Marília. **Cálculo A: funções, limite, derivação e integração**. 6. ed. São Paulo, SP: Pearson Education do Brasil, c2007. ix, 449 p., il. ISBN 9788576051152.

HUGHES-HALLETT, Deborah *et al.* **Cálculo e aplicações**. Colaboração de Otto K. Bretscher, Eric Connally, Richard D. Porter. Tradução de Elza Furtado Gomide. São Paulo, SP: Blücher, 1999. xii, 329 p., il. ISBN 9788521201786.

THOMAS, George B.; WEIR, Maurice D.; HASS, Joel. **Cálculo, vol. 1**. Tradução de Kleber Roberto Pedroso, Regina Célia Simille de Macedo. Revisão de Claudio Hirofume Asano. 12. ed. São Paulo, SP: Pearson Education do Brasil, 2012. xii, 634 p., il. ISBN 9788581430867.

GLOSSÁRIO

Aproximação linear	Representação local de uma função por uma reta tangente em torno de um ponto, usada para estimar valores próximos a esse ponto.
Assíntota	Reta cuja distância para a curva da função tende a zero à medida que x tende para o infinito positivo, ao infinito negativo ou a um ponto específico onde a função é indefinida.
Cálculo	Área da matemática que investiga propriedades locais e globais de funções reais a partir de limites, descrevendo quantitativamente com derivadas e integrais, respectivamente, como grandezas mudam e como essas mudanças se acumulam.
Concavidade	A curva voltada para cima (no formato da letra U) ou para baixo (como um $\cap$) no gráfico de uma função.
Continuidade	Propriedade de uma função cujo valor em um ponto coincide com o limite da função nesse ponto, indicando ausência de saltos ou rupturas.
Derivada	Taxa instantânea de variação de uma função; representa a inclinação da reta tangente em um ponto da função.
Derivada de ordem superior	Derivada obtida pela diferenciação sucessiva de uma função, como a derivada segunda, terceira e assim por diante.
Descontinuidade	Ponto em que o limite da função não coincide com seu valor, ou não existe.

Diferenciação	Processo de encontrar a derivada de uma função.
Diferenciação automática	Interpreta um programa como a composição de funções elementares e constrói, de forma algorítmica, o operador diferencial associado, obtendo o resultado via regra da cadeia.
Diferenciabilidade	Uma função é diferenciável em um ponto se sua derivada existe nesse ponto (não possui cantos agudos nem quebras).
Função	Relação matemática que associa a cada elemento de um conjunto de entrada exatamente um elemento de um conjunto de saída.
Função contínua	Função para a qual mudanças suficientemente pequenas na entrada (x) resultam em mudanças arbitrariamente pequenas na saída $f(x)$.
Limite	Valor ao qual uma função se aproxima quando a variável independente se aproxima de um determinado ponto.
Limite bilateral	Limite de uma função quando a variável se aproxima de um ponto pelos dois lados, direito e esquerdo.
Limite lateral	Limite de uma função quando a variável se aproxima de um ponto por apenas um dos lados, direito ou esquerdo.
Máximo/Mínimo absoluto	Maior/menor valor assumido por uma função em todo o seu domínio ou em um intervalo considerado.

Máximo/Mínimo local	Valor da função em um ponto que é maior/menor do que os valores assumidos em pontos próximos.
Otimização	Processo de encontrar valores máximos ou mínimos de uma função utilizando pontos críticos.
Ponto crítico	Ponto do domínio de uma função onde a derivada é nula ou não existe. Pode indicar um máximo ou mínimo local.
Ponto de inflexão	Ponto em que a concavidade do gráfico da função muda.
Singularidade	Ponto em que a expressão que define a função deixa de ser válida, ou por não estar definida ou por apresentar comportamento ilimitado, como divergência para infinito.
Taxa de variação média	Razão entre a variação da função e a variação da variável independente em um intervalo.
Taxa de variação instantânea	Taxa de variação em um ponto específico, definida pelo limite da taxa média e representada pela derivada.